OpenGL TD2

malek.bengougam@gmail.com

Coordonnées homogènes

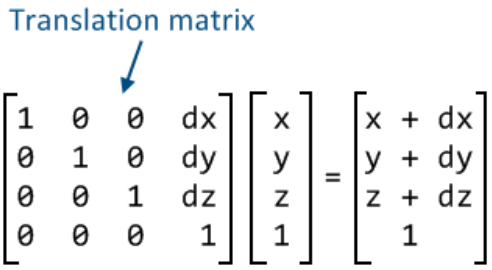
On connait deux types de transformations, les transformations linéaires (rotation, échelle uniforme, changement de base, ...) et affines (translations, projections …).

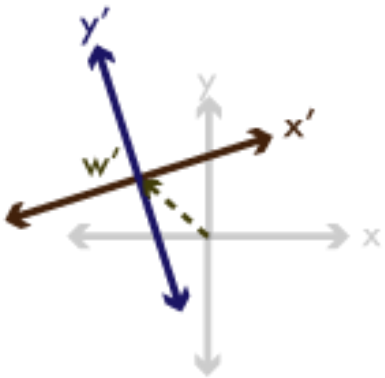
Prenons le cas d'une matrice de rotation 3x3 **R** et un vecteur ***t*** représentant une translation.
La transformation d'un vecteur v, avec un ordre rotation <u>puis</u> translation s'exprime ainsi :

$$\boldsymbol{v}' = \boldsymbol{t} + R * \boldsymbol{v}$$

Il est possible de combiner rotation et translation dans une seule et même matrice en ajoutant une dimension supplémentaire. On aura ainsi une matrice 4x4 (4 lignes, 4 colonnes) et donc 4 composantes par vecteur. L'équation s'exprime alors sous forme matricielle par combinaison (concaténation) des matrices rotation et translation (encore une fois, l'ordre est important) :

$$\boldsymbol{v}' = T * R * \boldsymbol{v}$$





```
mat4(vec4(xx, xy, xz, 0),
     vec4(yx, yy, yz, 0),
     vec4(zx, zy, zz, 0),
     vec4(wx, wy, wz, 1))
```

X', Y', Z' sont les axes, ou vecteurs, formant la base du nouveau repère (respectivement "right", "up" et "forward"), tandis que W' est ici la translation.

La dernière composante de chacune des colonnes est appelée coordonnée homogène car c'est par l'ajout de cette dimension supplémentaire que l'on peut combiner transformations affines et transformations linéaires.

Notez qu'en GLSL, `gl_Position` est un vec4 et non un vec3 car le GPU attend des coordonnées homogènes en sortie de la phase de Vertex Processing. La dernière coordonnée est nommée 'w'.

Interprétation de w

Une question se pose, dans le cas d'un vecteur ou d'un point, quelle doit être la valeur de w ?
La réponse simple est, w est égal à zéro (0) dans le cas d'un vecteur (direction / orientation) mais w est égal à un (1) dans le cas d'un point (vertex, position). Dans la matrice ci-dessus combinant transformation linéaire et translation, on remarque que seul le w de la quatrième colonne vaut un. Ce qui est logique vue que cette colonne exprime ici l'origine du repère, une position donc.

D'un point de vue pratique on remarque que la multiplication d'un vecteur directeur par une matrice ne doit modifier que son orientation ou sa magnitude.
En effet, un vecteur directeur n'a plus d'information d'origine, mais si une matrice contient une translation, cette translation va se combiner au vecteur directeur ce qui va produire un point et non plus une direction !
Dans ce cas il faudra veiller à ce que la coordonnée "w" d'un vecteur directeur vaut bien zéro, ou alternativement, ne pas utiliser de matrices homogènes avec un vecteur directeur.

Division perspective

L'ajout d'une dimension supplémentaire (4D) est pratique mais à un moment ou un autre il faudra repasser en 3D. C'est le rôle des matrices de projections. Cependant, dans le cas de la programmation d'un GPU, la projection ne s'effectue pas immédiatement mais en altérant la 4ième coordonnée. Voici un exemple de multiplication d'un vecteur 4D par une matrice de projection.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 4 \end{bmatrix}$$

La raison est qu'en interne, après une phase de clipping basée sur cette 4eme dimension "w" qui va éliminer la géométrie en dehors de la pyramide de vision, le GPU procède à une étape supplémentaire appelée la division perspective.

$$\frac{1}{4} * (2,3,4,4) = (0.5,0.75,1,1)$$

Cette étape est fondamentale afin que les coordonnées envoyées au *rasterizer* soient bien des valeurs entre [-1;+1] sur les 3 axes, ce qui forme nos fameuses coordonnées normalisées (NDC).

Ainsi NDC = `gl_Position.xyz / gl_Position.w`.

Ensuite, le rasterizer, va interpoler les coordonnées en NDC et générer un appel au fragment shader. Durant cette phase, les coordonnées 2D en repère NDC sont générées par la division de x et y en NDC par z en NDC.

Note : ne pas confondre la division perspective opérée par le GPU et la projection perspective.
Une matrice de projection (perspective ou non) tire toujours partie de ce mécanisme de division perspective mais ne produit pas automatiquement un effet de point de fuite.

En supplément, lire les 3 premiers chapitres de https://thebookofshaders.com/01/?lan=fr

Partie 1 – Transformations 2D

Exercice 1 - découverte des "uniforms"

Nous connaissons déjà les déclarations "**attribute**" et "**varying**" du GLSL.
Le GPU dispose d'une zone mémoire dédiée aux constantes, c'est-à-dire aux données qui vont rester invariante durant un appel de rendu (`glDrawArrays(), glDrawElements()` …).

Le mot clé "**uniform**" en GLSL permet de déclarer des variables dont le contenu peut être modifié depuis le programme client, ceci contrairement au mot-clé "const" qui indique une constante littérale.

Une "**uniform**" peut être déclarée dans un Vertex Shader, un Fragment Shader, ou les deux.
Exemple d'une déclaration :

```
uniform float u_time; // le préfixe u_ indique une "uniform" pour nous
```

On peut définir une valeur spécifique à une "uniform" à l'aide des fonctions commençant par `glUniform***`().

Dans notre cas, on doit affecté une valeur de type float, on utilisera donc la fonction `glUniform`**`1f`**().

De manière analogue aux "attribute"s il faut accéder au "location" de chaque "uniform" afin qu'OpenGL sache explicitement quelle "uniform" nous allons modifier. La fonction int glGetUniformLocation(program, "name") permet de récupérer cet identifiant.Exemple :

```
float currentTime = (float)glfwGetTime();
Int timeLocation = glGetUniformLocation(program,  "u_time");
glUniform1f(timeLocation, currentTime);
```

Il est important de bien utiliser la fonction correspondant avec le type déclaré dans le glsl, ainsi on utilisera

- **glUniform1i**(program, valeur) pour les int ainsi que les sampler2D/3D/Cube
- **glUniform3f**(program, valeur1, valeur2, valeur3) ou **glUniform3f**(program, pointeur) pour les vec3, où les valeurs sont des float.
- **glUniformMatrix4fv**() pour les matrices mat4, dont les valeurs sont des float (4x4 = 16 float)s.

Exercice 2 – rotations

Appliquez une rotation autour de l'axe forward (Z); Les rotations positives s'éffectue dans le sens trigonométrique (anti-horaire, counterclockwise)
Générez la matrice sous forme d'un tableau C et, après avoir récupéré le location du mat4 correspondant, utilisez la fonction **glUniformMatrix4fv**() pour affecter la valeur au mat4.

Créez une structure **mat4** en C contenant un tableau de *float* représentant les colonnes de la matrice.

Ne pas oublier que la représentation mathématique d'une matrice est en colonne, et qu'OpenGL attend qu'on lui fournisse la définition de la matrice COLONNE après COLONNE (on dit qu'OpenGL est colonne d'abord, "column-major").
En mémoire, ou dans un tableau C, les colonnes apparaissent linéarisées (en ligne) mais sont interprétées comme des colonnes par OpenGL.

Exercice 3 – facteur d'échelle et projection orthographique

Appliquez un scale uniforme de 100 unités à travers une autre mat4. Il est possible et recommandé d'appliquer à la fois le scale et la rotation précédente, le scale s'appliquant en premier sur le vecteur (en partant de la droite).

Afin de visualiser correctement votre objet, qui est maintenant 100 fois trop grand pour le repère NDC, il faut procéder à une projection. Une projection, sous forme de mat4, s'applique toujours en dernier, c'est donc toujours la matrice la plus à gauche dans une multiplication, exemple :

```
gl_Position = u_projectionMatrix * … * vec4(a_position, 1.0);
```

Nous allons considérer que nos unités sont des pixels. La projection doit donc convertir les pixels de notre fenêtre vers les coordonnées NDC. On connait déjà les dimensions de notre fenêtre par les valeurs retournées par la fonction **glfwGetWindowSize**().

Cette forme de projection est simplement un scale mais avec un facteur inverse de 1.0/width et 1.0/height respectivement pour les facteurs d'échelles de l'axe "right" et l'axe "up".

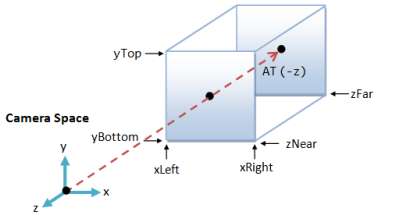
Notez que la projection est ici centrale, c'est à dire que l'origine reste au centre de l'écran.

***Note** : la projection est une opération différente de la conversion effectuée par glViewport().*
Une projection dans notre contexte, converti des unités du repère monde de l'utilisateur vers le repère NDC du GPU, tandis que glViewport() converti le NDC vers le repère de la fenêtre.

Exercice 4 - projection orthographique générique

Implémentez la matrice de projection orthographique générique en créant une fonction ortho(mat4* m, float left, float right, float bottom, float top, float znear, float zfar)

$$P = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -\frac{right+left}{right-left} \\ 0 & \frac{2}{top-bottom} & 0 & -\frac{top+bottom}{top-bottom} \\ 0 & 0 & \frac{-2}{far-near} & -\frac{far+near}{far-near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Appliquez cette matrice de sorte à ce que l'origine soit en haut à gauche, l'objet doit dès lors s'afficher en haut à gauche de l'écran et centré à l'origine.

Appliquez également une matrice de translation afin de déplacer la position de votre objet à l'écran, les unités étant toujours des pixels.

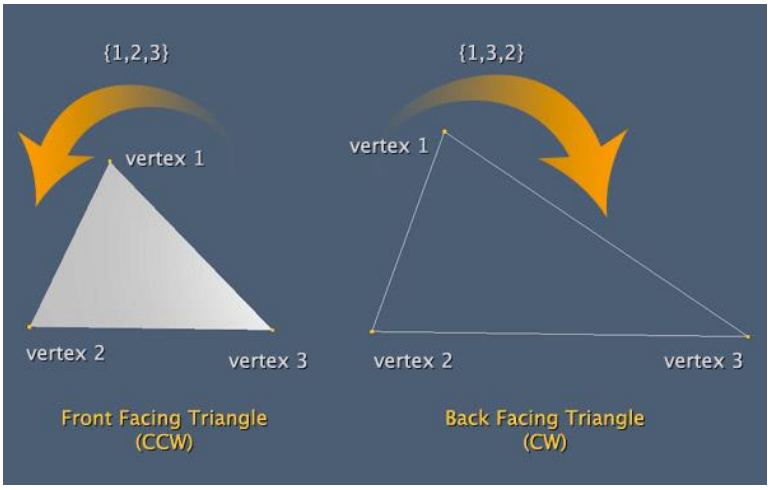
Partie 2 – Rasterization

2.1 avant la rasterization

Les étapes précédant la rasterization sont des étapes de test. Les primitives peuvent être testées par le GPU afin de déterminer si elles sont visibles ou non.

Une étape de ces étapes est contrôlable, c'est le "*backface culling*" ou suppression des faces cachées. Par défaut le "*culling*" est désactivé par OpenGL. Pour l'activer il faut utiliser la fonction **glEnable**() en spécifiant l'état **GL_CULL_FACE**.
OpenGL étant une machine à état explicite, l'état reste alors actif tant que le programmeur n'a pas modifié l'état ou désactivé l'état avec un appel à **glDisable**(**GL_CULL_FACE**);

La suppression d'une face dépend de l'ordonnancement des vertices (polygon winding). Le programmeur peut spécifier quel ordonnancement produit des faces visibles, par défaut OpenGL n'affiche que les faces dont les vertices sont spécifiés dans le sens anti-horaire (counter-clockwise).



La fonction **glFrontFace**() permet d'indiquer quel ordonnancement de face sera considéré comme visible (*front face*).

- **GL_CCW**, *counter-clockwise*, valeur par défaut, seules les faces anti-horaires sont visibles
- **GL_CW**, *clockwise*, seules les faces horaires sont visibles

Il est également possible de spécifier quel type de face est visible à l'aide de la fonction **glCullFace**(). Les faces-avants (par défaut), les faces-arrières (pour visualiser l'intérieur d'un objet par exemple), ou bien les deux (revient à désactiver le *culling*).

- **GL_FRONT** : affiche les faces-avants
- **GL_BACK** : affiche les faces-arrières
- **GL_FRONT_AND_BACK** : affiche toutes les faces

Exercice 5 – back face culling

Activez le "*face culling*" de sorte à n'afficher que les faces-avants définies dans le sens anti-horaire. Comme nous ne sommes pas en 3D ici, ce test va simplement permettre de vérifier le bon ordonnancement de vos triangles.